PRODUCT REQUIREMENTS DOCUMENT

Multi-Agent Financial Research System

An AI-Powered, Source-Grounded Financial Document Intelligence Platform

Prepared for: Infosys Springboard Virtual Internship | Sponsor: Vidzai Digital

Document Type: Product Requirements Document (PRD)

Version 1.0 | July 2026

Table of Contents

1. Document Control

1.1 Document Information

Field	Details
Document Title	Product Requirements Document — Multi-Agent Financial Research System
Project Sponsor	Vidzai Digital
Program	Infosys Springboard Virtual Internship 2026
Document Owner	Product Manager / Team Lead
Team Size	5 Students
Version	1.0
Status	Draft — Ready for Team Review
Classification	Confidential Information
Last Updated	July 2026

1.2 Revision History

Version	Date	Author	Description of Change
0.1	Week 1	Product Team	Initial draft created from project brief
0.5	Week 1	Product Team	Added multi-agent architecture and tech stack
1.0	Week 2	Product Team	Finalized PRD, circulated for team sign-off

1.3 Approvals

Role	Name	Status	Date
Project Sponsor	Vidzai Digital Reviewer	Pending	—
Team Lead	TBD	Pending	—
Engineering Lead	TBD	Pending	—
QA Lead	TBD	Pending	—

2. Executive Summary

The Multi-Agent Financial Research System is an AI-powered financial document intelligence platform that allows users to upload company financial reports — annual reports, 10-K filings, balance sheets, income statements, cash flow statements, MD&A sections, and auditor reports — and receive fast, accurate, source-grounded answers to natural-language financial questions.

Rather than manually reading hundreds of pages of dense financial disclosure, a user can upload a report (or use the pre-loaded seed company documents provided out of the box) and ask questions such as "What is the company's revenue?", "Compare TCS and Infosys profitability," or "Identify the major financial risks." The system responds using Retrieval-Augmented Generation (RAG), grounding every answer in the source document and citing the exact page it was drawn from.

At the core of the platform is a team of six specialized AI agents — Document, Extraction, Red Flag, Comparison, Research, and Report — that collaborate in a coordinated pipeline to parse documents, extract structured financial metrics, detect risk indicators, benchmark companies, answer research questions, and compile a professional analyst-style PDF report. The system is built with a strict non-hallucination guarantee: every claim the platform makes must be traceable to an uploaded source document.

This PRD defines the complete product scope, functional and non-functional requirements, multi-agent architecture, data model, API design, user stories, acceptance criteria, success metrics, risks, and delivery milestones needed for a development team of five students to design, build, test, and deploy the system over an eight-week internship engagement.

3. Vision Statement

To become the fastest, most trustworthy way for students, analysts, and finance professionals to understand a company's financial health — replacing hours of manual document reading with minutes of grounded, cited, AI-assisted research, without ever sacrificing factual accuracy for speed.

We envision a platform where a finance student can upload a 300-page annual report and, within seconds, ask multi-part questions and receive step-by-step reasoned answers backed by exact citations — effectively experiencing how a professional equity research desk operates, augmented by a coordinated team of specialized AI agents rather than a single generic chatbot.

4. Problem Statement

Finance students, MBA candidates, and early-career analysts find it difficult and time-consuming to read and understand lengthy company annual reports, financial statements, and regulatory filings. Manually extracting key metrics, identifying financial risks, and comparing companies across multiple documents requires significant effort, domain expertise, and time that most learners and junior professionals do not have.

4.1 Core Pain Points

- Financial reports routinely run 150 to 400 pages, making manual review slow and error-prone.
- Key metrics (revenue, EBITDA, margins, debt) are scattered across statements, notes, and MD&A sections rather than centralized.
- Identifying red flags — declining margins, rising debt, auditor qualifications — requires trained financial judgement that many students lack.
- Comparing two or more companies side by side means manually cross-referencing multiple lengthy documents.
- Generic AI chatbots hallucinate financial figures when not strictly grounded in source documents, which is unacceptable in a financial context.
- There is no single tool that combines document understanding, structured extraction, risk detection, comparison, conversational Q&A, and report generation in one workspace.

5. Goals

The following goals define what the system must achieve to be considered successful at the end of the internship engagement.

1. Build a multi-agent AI system where six specialized agents collaborate to analyze real company financial documents with strict source grounding.
2. Automatically extract key financial metrics, ratios, and performance indicators from uploaded filings via a dedicated Extraction Agent.
3. Detect and surface financial red flags, anomalies, and risk indicators without requiring the user to prompt for them, via a dedicated Red Flag Agent.
4. Answer multi-part financial research questions with step-by-step reasoning and exact page-level source citations, via a Research Agent.
5. Generate structured, analyst-style research reports exportable as a professional PDF, via a Report Agent.
6. Guarantee that the system never generates financial information that is not grounded in an uploaded source document.

6. Objectives

Objectives translate the goals above into measurable engineering targets for the eight-week build.

Objective	Target
Document ingestion reliability	≥ 95% of uploaded PDFs parse successfully with page numbers and metadata preserved
Extraction coverage	Extraction Agent correctly identifies core metrics (Revenue, Net Profit, EBITDA, EPS, ROE, ROCE, Debt) in ≥ 90% of standard annual report formats
Citation accuracy	≥ 95% of Research Agent answers include a correct, verifiable page citation
Hallucination rate	0% of answers contain financial figures that cannot be traced to a source document
Response latency	Research Agent answers returned in under 8 seconds for a single-document query
Report generation	Report Agent produces a complete, readable PDF report in under 30 seconds
Seed data readiness	System pre-loaded with 3–4 seed company annual reports for immediate demo use
Team delivery	All six agents, orchestration layer, and frontend delivered within the 8-week milestone plan

7. Scope

This section defines the boundary of the Multi-Agent Financial Research System for the internship delivery. The scope is deliberately constrained to a sequential, six-agent pipeline operating over user-uploaded and pre-loaded seed financial documents, so that a five-person student team can realistically design, build, and demonstrate the full product within eight weeks.

8. In Scope

- User authentication (register, login, profile) using JWT-based sessions.
- Research workspace creation and management — each workspace holds documents, chat history, extracted metrics, and generated reports.
- Upload and processing of financial PDFs: annual reports, 10-K filings, balance sheets, income statements, cash flow statements, MD&A, and auditor reports.
- Document processing pipeline: PDF parsing, optional OCR, text cleaning, chunking, metadata generation, embedding generation, and vector storage.
- Six specialized AI agents: Document, Extraction, Red Flag, Comparison, Research, and Report.
- Sequential multi-agent orchestration triggered automatically on document upload and on user query.
- Conversational, citation-backed question answering grounded strictly in uploaded documents (RAG).
- Multi-company financial comparison and benchmarking.
- Automated red-flag / risk detection surfaced without explicit user prompting.
- Structured, downloadable PDF report generation (Executive Summary, Financial Overview, Risks, Comparison, Outlook).
- Pre-loaded seed company documents (3–4 companies) available at launch for immediate demonstration.
- A React/Next.js web frontend covering dashboard, upload, chat, comparison, and report views.

9. Out of Scope

- Real-time stock price or market data integration.
- Direct SEC / regulatory filing ingestion via API (documents are user-uploaded PDFs only).
- Financial forecasting, valuation modelling, or price-target generation.
- Portfolio construction or investment recommendation features.
- Multi-language report generation (English only for v1.0).
- Voice-based interaction or voice assistant interface.
- Native mobile applications (iOS / Android).
- Enterprise SSO, multi-tenant billing, or role-based organization admin consoles.
- Support for non-financial document types (contracts, resumes, legal filings, etc.).

10. Stakeholders

Stakeholder	Role	Interest
Vidzai Digital	Project Sponsor	Defines project brief, evaluates milestones and final deliverable
Infosys Springboard	Internship Program Owner	Provides internship framework, evaluation criteria, and certification
Student Development Team (5)	Builders	Design, build, test, and demonstrate the system across 8 weeks
Financial Analysts / Students (End Users)	Primary Users	Use the platform to research and understand company financials faster
Academic Mentors	Evaluators	Assess technical quality, correctness, and completeness of the build

11. User Personas

11.1 Priya — Finance Student

Attribute	Detail
Background	Final-year B.Com student preparing for equity research interviews
Goal	Quickly understand a company's revenue trend, profitability, and key risks before a case study discussion
Frustration	Annual reports are 300+ pages; she doesn't know where to find specific numbers
How the product helps	Uploads the annual report, asks direct questions, gets cited answers in seconds

11.2 Rohan — MBA Candidate

Attribute	Detail
Background	First-year MBA student building a comparative analysis for a finance elective
Goal	Compare two companies' margins, debt, and ROE side by side for a class assignment
Frustration	Manually cross-referencing multiple PDFs is slow and error-prone
How the product helps	Uploads both companies' reports and uses the Comparison Agent for an instant benchmarking table

11.3 Ananya — Early-Career Analyst

Attribute	Detail
Background	Junior analyst at a boutique research firm, six months into her first role
Goal	Produce a quick first-pass research note flagging financial risks before her senior reviews it
Frustration	Needs a defensible, source-cited starting point — not a confident-sounding guess
How the product helps	Red Flag Agent surfaces anomalies automatically; Report Agent produces a structured PDF draft

12. User Journey

The primary end-to-end journey a user takes through the platform, from first login to receiving a finished research report.

7. User registers or logs in to the platform.
8. User creates a new Research Workspace (or opens an existing one, or starts from a pre-loaded seed company).
9. User uploads one or more financial documents (PDF).
10. The Document Agent parses, chunks, embeds, and indexes the document automatically.
11. The Extraction Agent and Red Flag Agent run automatically in the background and surface key metrics and warnings.
12. User opens the Chat interface and asks natural-language financial questions.
13. The Research Agent retrieves relevant chunks and returns a grounded, cited answer.
14. If multiple companies are uploaded, the user requests a comparison; the Comparison Agent generates a benchmarking table.
15. User clicks "Generate Report"; the Report Agent compiles all agent outputs into a structured PDF.
16. User downloads the report and/or continues the conversation for deeper research.

13. Product Features

13.1 User Authentication

- Login with email and password.
- Registration with basic profile details.
- User profile view and edit.
- JWT-based session management.

13.2 Research Workspace

- Users can create multiple named research workspaces.
- Each workspace stores its own uploaded documents, chat history, generated reports, extracted metrics, and agent outputs.
- Workspaces can be renamed, archived, or deleted.

13.3 Financial Document Upload

- Supports PDF uploads including annual reports and individual financial statements.
- System uploads, stores, reads, and extracts text from the document.
- Page numbers and document metadata are preserved throughout the pipeline.
- 3–4 pre-loaded seed company documents are available for immediate use without requiring an upload.

13.4 Document Processing Pipeline

- PDF parsing using PyMuPDF.
- Optional OCR support (Tesseract) for scanned documents.
- Text cleaning to remove noise, headers, footers, and artifacts.
- Chunking of cleaned text into retrievable segments.
- Metadata generation: document name, page number, section, and chunk ID.
- Embedding generation for every chunk.
- Vector storage in the vector database for semantic retrieval.

13.5 Vector Database

- Stores document embeddings with associated metadata.
- Supports semantic (similarity) search across all indexed chunks in a workspace.

13.6 Multi-Agent AI System

- Document Agent — parsing, chunking, embedding, and indexing.
- Extraction Agent — structured financial metric extraction.
- Red Flag Agent — anomaly and risk detection.
- Comparison Agent — multi-company benchmarking.

- Research Agent — conversational, cited question answering.
- Report Agent — PDF research report compilation.

13.7 Conversational Research Interface

- Chat-style UI for asking financial questions.
- Multi-part, multi-step questions are supported with step-by-step reasoning.
- Every answer displays its exact source citation (document name and page number).
- Chat history is saved per workspace.

13.8 Company Comparison

- Side-by-side comparison of revenue, profit, margins, debt, and financial ratios across uploaded companies.
- Benchmarking tables generated automatically from extracted metrics.

13.9 Report Generation

- One-click generation of a structured, downloadable PDF report.
- Report includes Executive Summary, Financial Overview, Company Performance, Risks, Strengths, Weaknesses, Financial Ratios, AI Insights, Comparative Analysis, and Final Conclusion.

14. Functional Requirements

Functional requirements are grouped by module below. Each requirement carries a unique identifier for traceability to test cases and user stories.

14.1 Authentication

ID	Requirement
FR-AUTH-01	System shall allow a new user to register with name, email, and password.
FR-AUTH-02	System shall allow a registered user to log in and receive a JWT session token.
FR-AUTH-03	System shall allow a user to view and update their profile.
FR-AUTH-04	System shall reject invalid credentials with a clear error message.

14.2 Workspace

ID	Requirement
FR-WS-01	System shall allow a user to create a new research workspace with a name.
FR-WS-02	System shall list all workspaces belonging to the logged-in user.
FR-WS-03	System shall allow a user to delete a workspace and its associated data.
FR-WS-04	System shall isolate documents, chats, and reports per workspace.

14.3 Document Upload

ID	Requirement
FR-DOC-01	System shall accept PDF file uploads up to a defined size limit.
FR-DOC-02	System shall store the uploaded file and return a success confirmation.
FR-DOC-03	System shall reject non-PDF or corrupted files with a descriptive error.
FR-DOC-04	System shall associate every uploaded document with a workspace and owner.

14.4 PDF Parsing and Chunking

ID	Requirement
FR-PARSE-01	System shall extract raw text from every page of an uploaded PDF.
FR-PARSE-02	System shall preserve the page number for every extracted text segment.
FR-PARSE-03	System shall clean extracted text, removing headers, footers, and artifacts.
FR-PARSE-04	System shall chunk cleaned text into retrievable segments with defined overlap.

ID	Requirement
FR-PARSE-05	System shall optionally apply OCR when a page contains no extractable text layer.

14.5 Embedding and Vector Search

ID	Requirement
FR-EMB-01	System shall generate an embedding vector for every text chunk.
FR-EMB-02	System shall store each embedding with document name, page, section, and chunk ID metadata.
FR-EMB-03	System shall retrieve the top-k most semantically similar chunks for a given query.

14.6 Agents

ID	Requirement
FR-AGT-01	System shall automatically trigger the Document Agent on every successful upload.
FR-AGT-02	System shall automatically trigger the Extraction Agent once a document is indexed.
FR-AGT-03	System shall automatically trigger the Red Flag Agent once metrics are extracted.
FR-AGT-04	System shall trigger the Comparison Agent when two or more companies exist in a workspace.
FR-AGT-05	System shall trigger the Research Agent whenever a user submits a chat question.
FR-AGT-06	System shall trigger the Report Agent when the user requests report generation.
FR-AGT-07	Every agent response shall include a reference to its supporting source chunk(s).

14.7 Chat

ID	Requirement
FR-CHAT-01	System shall accept free-text financial questions from the user.
FR-CHAT-02	System shall return an answer grounded only in retrieved document chunks.
FR-CHAT-03	System shall display the document name and page number for every cited answer.
FR-CHAT-04	System shall persist chat history per workspace and allow the user to revisit it.
FR-CHAT-05	System shall decline to answer, rather than guess, when no relevant chunk is found.

14.8 Comparison

ID	Requirement
FR-CMP-01	System shall generate a comparison table of key metrics across selected companies.

ID	Requirement
FR-CMP-02	System shall indicate the stronger or weaker performer per metric where applicable.

14.9 Report and PDF Export

ID	Requirement
FR-RPT-01	System shall compile outputs from all agents into a structured report object.
FR-RPT-02	System shall render the report as a downloadable, professionally formatted PDF.
FR-RPT-03	System shall include Executive Summary, Key Financials, Red Flags, Comparison, and Outlook in every report.
FR-RPT-04	System shall allow the user to regenerate a report after new documents are added.

14.10 History and Search

ID	Requirement
FR-HIST-01	System shall allow a user to view past chat sessions within a workspace.
FR-HIST-02	System shall allow a user to view and re-download previously generated reports.

14.11 Settings

ID	Requirement
FR-SET-01	System shall allow a user to update account settings and log out.
FR-SET-02	System shall allow a user to delete an uploaded document and its associated embeddings.

15. Non-Functional Requirements

Category	Requirement
Performance	Research Agent shall return an answer within 8 seconds for a single-document query and within 15 seconds for a multi-document comparison query.
Security	All user passwords shall be hashed; all API endpoints (except login/register) shall require a valid JWT; uploaded documents shall be accessible only to their owning workspace.
Scalability	The vector database and document store shall support at least 50 concurrent workspaces and 500 indexed documents without degradation for the internship demo environment.
Reliability	The document processing pipeline shall have a success rate of at least 95% across standard PDF annual reports.
Availability	The application shall target 99% uptime during the demo and evaluation period.
Latency	Chunk retrieval from the vector database shall complete in under 1 second for a typical workspace.
Usability	A first-time user shall be able to upload a document and receive an answered question within 3 minutes without external guidance.
Maintainability	Each agent shall be implemented as an independently testable module with a clearly defined input/output contract.
Extensibility	The orchestration layer shall allow new agents to be added to the pipeline without modifying existing agent code.
Data Integrity	Extracted metrics and citations shall remain consistent with the source document even after report regeneration.
Accuracy / Grounding	The system shall never present a financial figure that cannot be traced to a specific page in an uploaded source document.
Observability	Every agent execution shall be logged with input, output, and duration for debugging and evaluation.

16. Multi-Agent Architecture

The system contains six specialized AI agents that collaborate in a mostly sequential pipeline, coordinated by an orchestration layer. Each agent has a narrow, well-defined responsibility and a clear input/output contract, and is independently testable — similar to how a hospital routes a patient through reception, doctor, lab, radiologist, and pharmacist rather than relying on one generalist to do everything.

16.1 Agent 1 — Document Agent

Input	Output	Key Responsibilities
PDF (raw upload)	Chunks and embeddings, indexed in the vector database	Receive the uploaded PDF; Parse the PDF and extract raw text; Clean the extracted text; Chunk the document into retrievable segments; Generate embeddings for each chunk; Store vectors in the vector database; Save page, section, and chunk metadata

- Receive the uploaded PDF
- Parse the PDF and extract raw text
- Clean the extracted text
- Chunk the document into retrievable segments
- Generate embeddings for each chunk
- Store vectors in the vector database
- Save page, section, and chunk metadata

16.2 Agent 2 — Extraction Agent

Input	Output	Key Responsibilities
Indexed chunks	Structured financial metrics	Extract Revenue, Net Profit, EBITDA, and EPS; Extract ROE, ROCE, Debt, Assets, and Liabilities; Extract Cash Flow, Operating Margin, and Gross Margin; Extract disclosed market and business risks; Store the extracted values as structured financial records

- Extract Revenue, Net Profit, EBITDA, and EPS
- Extract ROE, ROCE, Debt, Assets, and Liabilities
- Extract Cash Flow, Operating Margin, and Gross Margin
- Extract disclosed market and business risks
- Store the extracted values as structured financial records

16.3 Agent 3 — Red Flag Agent

Input	Output	Key Responsibilities
Extracted metrics	Financial warnings	Detect declining revenue trends; Detect increasing debt levels; Detect negative cash flow; Detect auditor qualifications; Detect litigation and business risks; Detect high liabilities and margin decline; Generate a structured list of warnings with severity

- Detect declining revenue trends
- Detect increasing debt levels
- Detect negative cash flow
- Detect auditor qualifications
- Detect litigation and business risks
- Detect high liabilities and margin decline
- Generate a structured list of warnings with severity

16.4 Agent 4 — Comparison Agent

Input	Output	Key Responsibilities
Metrics from multiple companies	Comparison tables	Cross-reference financial data across uploaded companies; Generate revenue, profit, and margin comparisons; Generate debt and financial ratio comparisons; Produce a benchmarking table for the Report Agent

- Cross-reference financial data across uploaded companies
- Generate revenue, profit, and margin comparisons
- Generate debt and financial ratio comparisons
- Produce a benchmarking table for the Report Agent

16.5 Agent 5 — Research Agent

Input	Output	Key Responsibilities
User's natural-language question	Grounded answer with citations	Accept single-part or multi-part user questions; Retrieve the most relevant chunks from the vector database; Apply Retrieval-Augmented Generation (RAG) to compose an answer; Reason step-by-step for multi-part questions; Attach exact page-level citations to every claim; Decline to answer when no grounded evidence is found

- Accept single-part or multi-part user questions
- Retrieve the most relevant chunks from the vector database
- Apply Retrieval-Augmented Generation (RAG) to compose an answer
- Reason step-by-step for multi-part questions
- Attach exact page-level citations to every claim
- Decline to answer when no grounded evidence is found

16.6 Agent 6 — Report Agent

Input	Output	Key Responsibilities
Outputs of all other agents	Downloadable, structured PDF report	Compile the Executive Summary; Compile the Financial Overview and Key Financials; Compile Company Performance and Comparative Analysis; Compile Risks, Strengths, and Weaknesses; Compile Financial Ratios and AI Insights; Compile the Final Conclusion; Render the final PDF for download

- Compile the Executive Summary
- Compile the Financial Overview and Key Financials
- Compile Company Performance and Comparative Analysis
- Compile Risks, Strengths, and Weaknesses
- Compile Financial Ratios and AI Insights
- Compile the Final Conclusion
- Render the final PDF for download

16.7 Agent Orchestration Flow

The orchestration layer decides which agent runs, in what order, and what data is passed between agents. For the internship build, the pipeline is intentionally sequential rather than a fully dynamic graph, since the workflow does not require agents to call each other back and forth.

17. User uploads PDF.
18. Document Agent runs — parses, chunks, embeds, and indexes the document.
19. Extraction Agent runs — pulls structured financial metrics.
20. Red Flag Agent runs — scans extracted metrics for warnings.
21. Results are stored in the database.
22. User asks a question — Research Agent runs, retrieving chunks and generating a cited answer.
23. If a comparison is requested and two or more companies exist — Comparison Agent runs.
24. User clicks Generate Report — Report Agent runs, compiling all prior agent outputs into a PDF.

Why a Sequential Pipeline (LangChain) Rather Than a Dynamic Graph (LangGraph)

LangChain is well suited to building a single specialized agent — prompt templates, document loaders, retrieval, and chains connecting a question to a grounded answer. LangGraph, built on top of LangChain, is designed for multi-agent workflows where agents need to call each other dynamically and pass control back and forth, forming a graph rather than a straight line.

Because this project's workflow is almost entirely linear — Document → Extraction → Red Flag → Comparison → Research → Report — it can be implemented using LangChain alone, with the orchestration layer calling each agent function in sequence from Python code. LangGraph becomes valuable only if a future iteration requires an agent (for example, Research) to dynamically call back into another agent (for example, Document or Comparison) mid-reasoning. This PRD recommends LangChain for the v1.0 build, with LangGraph noted as a future-scope upgrade path.

17. RAG Workflow

Retrieval-Augmented Generation (RAG) is the mechanism that guarantees every answer is grounded in an uploaded source document rather than the language model's own unverified memory. This is the single most important technical concept in the product, since financial information must never be hallucinated.

17.1 Without RAG (Not Used in This Product)

- User Question → LLM → Answer (ungrounded; risk of hallucination).

17.2 With RAG (Used in This Product)

- User Question → Search Documents → Retrieve Relevant Pages → LLM → Grounded Answer with Citation.

17.3 Embeddings

Documents cannot be searched efficiently as raw text, so every chunk is converted into a numerical vector representation called an embedding. Chunks with similar meaning produce similar vectors, which enables semantic (meaning-based) search rather than simple keyword matching. For example, the sentence "The company's revenue increased" is converted into a high-dimensional vector such as [0.42, -0.15, 0.81, ...].

17.4 Vector Database Retrieval

Each chunk is stored in the vector database together with its embedding and metadata (document name, page number, section, chunk ID). When a user asks a question such as "What was revenue growth?", the system converts the question into an embedding and retrieves the closest matching chunks by vector similarity.

17.5 Grounded Answer Generation

The retrieved chunks are passed to the LLM together with the user's question. The LLM is instructed to answer strictly from the provided chunks and to cite the exact page each fact came from. If no relevant chunk is retrieved, the Research Agent must decline to answer rather than guess.

18. System Architecture

At a high level, the entire product is one pipeline: the user uploads a financial report, the document processing layer prepares it, the vector database stores it, the multi-agent AI system analyzes it, and the result is delivered back to the user as chat answers, comparisons, red flags, and a PDF report.

18.1 High-Level Pipeline

```
USER

↓ Upload Financial Report

Document Processing Layer

↓

Vector Database Storage

↓

Multi-Agent AI System

↓

Chat + PDF Report Output
```

18.2 Layered Architecture

```
React Frontend (Next.js, Tailwind CSS)

↓

FastAPI Backend (REST API layer)

↓

MongoDB (Users, Workspaces,      ChromaDB (Document

Chats, Metadata, Reports)         Embeddings)

↓

LangChain Orchestration Layer

↓

Document Agent  →  Extraction Agent  →  Red Flag Agent

↓

Comparison Agent  →  Research Agent  →  Report Agent

↓

PDF Report + Chat Response
```

18.3 Technology Stack

Layer	Recommended Technology
Frontend	React + Next.js + Tailwind CSS
Backend	FastAPI (Python)
Authentication	JWT (JSON Web Tokens)
Agent / LLM Framework	LangChain (sequential pipeline; LangGraph as a future upgrade)
LLM	Gemini API (preferred) or OpenAI API (optional)
PDF Parsing	PyMuPDF
OCR (optional)	Tesseract
Embeddings	Gemini / OpenAI Embeddings or Sentence Transformers
Vector Database	ChromaDB (or MongoDB Atlas Vector Search as an alternative)
Application Database	MongoDB
PDF Report Generation	ReportLab or WeasyPrint
Deployment — Frontend	Vercel
Deployment — Backend	Render (or Railway)

19. Database Design

The application uses MongoDB as the primary application database because financial reports vary widely in structure, and a flexible, document-oriented schema handles this variability better than a rigid relational schema. Vector embeddings are stored separately in ChromaDB, since MongoDB (outside of Atlas Vector Search) is not itself a vector database.

19.1 MongoDB Collections

Collection	Purpose	Key Fields
Users	Stores registered user accounts	_id, name, email, passwordHash, createdAt
Workspaces	Stores each user's research workspaces	_id, userId, workspaceName, createdAt
Documents	Stores metadata for every uploaded document	_id, workspaceId, filename, totalPages, uploadedAt
FinancialMetrics	Stores structured metrics extracted by the Extraction Agent	documentId, revenue, netProfit, ebitda, eps, roe, roce, debt, cashFlow
RedFlags	Stores warnings generated by the Red Flag Agent	documentId, flagType, description, severity, sourcePage
ChatHistory	Stores conversational Q&A per workspace	workspaceId, userQuestion, aiAnswer, citations, timestamp
Reports	Stores generated report metadata and file references	_id, workspaceId, reportUrl, generatedAt
AgentLogs	Stores execution logs for every agent run	agentName, workspaceId, input, output, durationMs, timestamp

19.2 Example Document Shapes

```
Users            { _id, name, email }

Workspaces       { _id, userId, workspaceName, createdAt }

Documents        { _id, workspaceId, filename, totalPages, uploadedAt }

FinancialMetrics { documentId, revenue, netProfit, debt, eps, roe }

ChatHistory      { workspaceId, userQuestion, aiAnswer, citations, timestamp }
```

19.3 Vector Database (ChromaDB)

ChromaDB stores the embedding for every document chunk together with its metadata, entirely separate from MongoDB. This two-database split (MongoDB for application data, ChromaDB for embeddings) is the recommended approach for the internship build because it is simple and beginner-friendly.

Field	Description
chunkId	Unique identifier for the chunk
embedding	Numerical vector representation of the chunk text
documentName	Name of the source document
page	Page number the chunk was extracted from
section	Section of the report the chunk belongs to (if identifiable)

Alternative: if the team uses MongoDB Atlas, MongoDB Atlas Vector Search can store both application data and embeddings in a single system, removing the need for a separate ChromaDB instance.

19.4 Recommended Folder Structure

A clear folder structure from day one saves significant time during a fast-paced, multi-person internship build.

```
Financial-Research-System/
├── frontend/
│   ├── app/                  (Next.js pages: dashboard, workspace, chat, report)
│   ├── components/           (UI components: uploader, chat window, tables)
│   ├── lib/                  (API client, auth helpers)
│   └── public/
├── backend/
│   ├── main.py                (FastAPI entry point)
│   ├── requirements.txt
│   ├── .env
│   ├── agents/
│   │   ├── document_agent.py
│   │   ├── extraction_agent.py
│   │   ├── redflag_agent.py
│   │   ├── comparison_agent.py
│   │   ├── research_agent.py
│   │   └── report_agent.py
│   ├── document_processing/
│   │   ├── pdf_parser.py
│   │   ├── ocr.py
│   │   ├── text_cleaner.py
│   │   └── chunker.py
│   ├── embeddings/
│   │   ├── embedding_service.py
│   │   └── vector_store.py
│   ├── database/
│   │   ├── mongo_client.py
│   │   └── models.py
│   ├── api/
│   │   ├── auth_routes.py
```

```
│   │   ├── workspace_routes.py
│   │   ├── upload_routes.py
│   │   ├── chat_routes.py
│   │   ├── compare_routes.py
│   │   └── report_routes.py
│   ├── services/
│   │   ├── orchestrator.py    (multi-agent pipeline coordination)
│   │   └── pdf_report_builder.py
│   ├── models/                (Pydantic schemas / data models)
│   └── utils/
├── docs/                      (PRD, architecture diagrams, meeting notes)
├── datasets/                  (3-4 pre-loaded seed company annual reports)
├── reports/                   (sample generated PDF reports)
└── README.md
```

20. API Design

REST APIs exposed by the FastAPI backend. All endpoints except registration and login require a valid JWT bearer token.

Method	Endpoint	Description
POST	/register	Create a new user account
POST	/login	Authenticate a user and return a JWT
GET	/profile	Retrieve the logged-in user's profile
POST	/workspace	Create a new research workspace
GET	/workspace	List all workspaces for the logged-in user
DELETE	/workspace/{workspaceId}	Delete a workspace and its associated data
POST	/upload	Upload a financial document to a workspace
GET	/documents/{workspaceId}	List documents uploaded to a workspace
DELETE	/documents/{documentId}	Delete a document and its embeddings
POST	/chat	Submit a question to the Research Agent
GET	/history/{workspaceId}	Retrieve chat history for a workspace
POST	/compare	Trigger the Comparison Agent across selected companies
GET	/metrics/{documentId}	Retrieve structured metrics extracted from a document
GET	/redflags/{documentId}	Retrieve red flags detected for a document
POST	/generate-report	Trigger the Report Agent and generate a PDF report
GET	/reports/{workspaceId}	List previously generated reports for a workspace
GET	/reports/{reportId}/download	Download a generated PDF report

21. User Stories

User stories are written in the standard Agile format: "As a [persona], I want [capability], so that [benefit]." Each story maps to one or more functional requirements defined in Section 14.

US-01 As a Financial Analyst, I want to register for an account, so that I can create and manage my own research workspaces.

US-02 As a Financial Analyst, I want to log in securely, so that my research and uploaded documents remain private.

US-03 As a Finance Student, I want to create a named research workspace, so that I can organize my research by company or project.

US-04 As a Finance Student, I want to upload multiple annual reports, so that I can compare company performance.

US-05 As a MBA Student, I want to use pre-loaded seed company documents, so that I can explore the product immediately without uploading my own files.

US-06 As a Financial Analyst, I want to see confirmation that my document uploaded successfully, so that I know the system is ready for analysis.

US-07 As a Investment Researcher, I want to have the system extract text from my PDF automatically, so that I don't need to manually copy information.

US-08 As a Business Analyst, I want to have page numbers preserved during processing, so that I can trust the citations in later answers.

US-09 As a Finance Student, I want to have the system chunk long documents intelligently, so that retrieval returns precise, relevant sections.

US-10 As a Chartered Accountant, I want to have the system extract Revenue, Net Profit, and EBITDA automatically, so that I don't need to search the report manually.

US-11 As a Chartered Accountant, I want to have the system extract ROE, ROCE, and Debt figures, so that I can quickly assess financial health.

US-12 As a Investment Researcher, I want to see extracted metrics stored in a structured format, so that I can reference them across sessions.

US-13 As a Early-Career Analyst, I want to receive automatic red-flag warnings without asking for them, so that I don't miss critical risk indicators.

US-14 As a Early-Career Analyst, I want to see when debt is increasing or margins are declining, so that I can flag concerns to my team early.

US-15 As a Early-Career Analyst, I want to see auditor qualifications surfaced automatically, so that I understand any compliance concerns.

US-16 As a MBA Student, I want to compare two companies' revenue and profit side by side, so that I can complete my comparative analysis assignment.

US-17 As a MBA Student, I want to compare debt and financial ratios across companies, so that I can benchmark performance accurately.

US-18 As a Corporate Strategy Team Member, I want to generate a comparison table across more than two companies, so that I can evaluate competitive positioning.

US-19 As a Finance Student, I want to ask natural-language questions about a report, so that I don't need to read the entire document.

US-20 As a Finance Student, I want to ask multi-part questions and get step-by-step answers, so that I can understand complex financial relationships.

US-21 As a Investment Researcher, I want to see the exact page citation for every answer, so that I can verify the information myself.

US-22 As a Professor, I want to have the system decline to answer when it lacks evidence, so that I can trust it never fabricates figures.

US-23 As a Financial Analyst, I want to review my past chat history in a workspace, so that I can continue research across multiple sessions.

US-24 As a Business Analyst, I want to generate a professional PDF report with one click, so that I can share findings with my team quickly.

US-25 As a Business Analyst, I want to have the report include an Executive Summary and Key Financials, so that stakeholders get a quick overview.

US-26 As a Early-Career Analyst, I want to have the report include a Red Flags section, so that risk factors are visible at a glance.

US-27 As a Corporate Strategy Team Member, I want to have the report include a Comparative Analysis section, so that I can present benchmarking results directly.

US-28 As a Financial Analyst, I want to download and re-access previously generated reports, so that I don't need to regenerate them each time.

US-29 As a Researcher, I want to delete a document I no longer need, so that my workspace stays organized and relevant.

US-30 As a Finance Student, I want to update my profile information, so that my account details stay current.

US-31 As a Investment Researcher, I want to search across my chat history, so that I can quickly find a past answer.

US-32 As a MBA Student, I want to see a clear dashboard of all my workspaces, so that I can navigate my research efficiently.

22. Acceptance Criteria

Acceptance criteria are defined per feature area using a Given / When / Then structure so that QA can validate each capability objectively.

Feature	Acceptance Criteria
Document Upload	Given a valid PDF financial report, when the user uploads it, then the system stores the file, confirms success, and makes it available for processing within the workspace.
Document Processing	Given an uploaded PDF, when the Document Agent runs, then text is extracted, cleaned, chunked, embedded, and indexed with accurate page numbers for at least 95% of pages.
Extraction Agent	Given an indexed annual report, when the Extraction Agent runs, then Revenue, Net Profit, and at least four other core metrics are correctly identified and stored.
Red Flag Agent	Given extracted metrics showing a declining trend or anomaly, when the Red Flag Agent runs, then a corresponding warning is generated and stored with its source page.
Comparison Agent	Given two or more companies in a workspace, when the user requests a comparison, then a table showing revenue, profit, margin, and debt across all companies is generated.
Research Agent — Grounded Answer	Given a user question with relevant content in the uploaded document, when the Research Agent runs, then it returns an answer with a correct page citation within 8 seconds.
Research Agent — No Hallucination	Given a user question with no relevant content in the uploaded document, when the Research Agent runs, then it explicitly states it cannot find the answer rather than fabricating one.
Report Agent	Given a workspace with processed documents, when the user clicks Generate Report, then a complete PDF containing all required sections is produced within 30 seconds.
Authentication	Given valid registration details, when a user registers and logs in, then they receive a valid session and can access only their own workspaces.

23. Success Metrics

Measurable KPIs used to evaluate the product at the end of the internship engagement.

KPI	Target
Document upload success rate	≥ 95% of valid PDFs upload and process successfully
Answer accuracy	≥ 90% of Research Agent answers are factually correct against the source document
Citation accuracy	≥ 95% of citations point to the correct source page
Retrieval precision	≥ 90% of retrieved chunks are relevant to the user's question
Response time	≤ 8 seconds for single-document questions, ≤ 15 seconds for comparisons
PDF generation success	≥ 95% of report generation requests complete without error
Red flag relevance	≥ 85% of surfaced red flags are judged relevant by a reviewer
Hallucination rate	0% of answers present unsupported financial figures

24. Risks

24.1 Technical Risks

- Inconsistent PDF formatting across companies may reduce extraction accuracy.
- Scanned or image-based PDFs may require OCR, which can introduce text errors.
- Large documents (300+ pages) may increase processing time and cost.
- Orchestration bugs could cause an agent to run on incomplete or stale data.

24.2 Business / Program Risks

- Eight-week timeline is tight for a five-person student team building six agents plus a full-stack application.
- Scope creep (adding forecasting, voice, or multi-language features) could jeopardize the core deliverable.
- Uneven skill distribution across the team could bottleneck specific modules (e.g., PDF parsing, prompt engineering).

24.3 AI and Hallucination Risks

- LLMs may generate plausible-sounding but incorrect financial figures if not strictly grounded.
- Poor retrieval quality (irrelevant chunks) can lead to answers that are technically grounded but contextually wrong.
- Ambiguous user questions (e.g., "profit" without specifying gross, operating, or net) can lead to mismatched answers.

24.4 PDF Parsing Risks

- Tables and multi-column layouts in financial statements are historically difficult to parse accurately.
- Footnotes and restated figures may be misattributed to the wrong period if not carefully parsed.

24.5 Performance Risks

- Embedding generation and vector search may slow down as the number of indexed documents grows.
- Concurrent agent execution across many workspaces may require queuing to avoid API rate limits.

25. Assumptions

- Users will primarily upload text-based (not purely scanned/image) PDF financial reports.
- A Gemini or OpenAI API key with sufficient quota will be available to the team throughout development.
- 3–4 seed company annual reports will be sourced and cleared for use in the pre-loaded demo environment.
- The team has (or will acquire) basic familiarity with FastAPI, React, and LangChain fundamentals.
- Financial documents are primarily in English for the v1.0 release.
- Evaluation will occur in a controlled demo environment rather than at large-scale production traffic.

26. Constraints

- Delivery window is fixed at 8 weeks as defined by the Infosys Springboard internship program.
- Team size is fixed at 5 students, requiring careful task distribution across agents and modules.
- LLM API usage is subject to rate limits and token costs, which constrains prompt size and retrieval depth.
- Deployment budget is limited to free or low-cost tiers of Vercel, Render/Railway, and the chosen vector database.
- The system must never hallucinate financial information — this is a hard product constraint, not a stretch goal.

27. Milestones

The delivery plan follows a four-milestone, eight-week structure, consistent with the official Infosys project brief.

27.1 Milestone 1 — Weeks 1–2: Foundation

- Study financial document structures (annual reports, 10-K filings, earnings transcripts) and finalize system architecture.
- Design multi-agent architecture, orchestration flow, agent responsibilities, and data models.
- Develop research workspace and session management — users can create named research sessions and manage uploaded documents.
- Build the Document Agent — upload, parsing, chunking, embedding generation, and vector database indexing, with 3–4 pre-loaded seed company documents.

27.2 Milestone 2 — Weeks 3–4: Extraction and Risk Detection

- Build the Extraction Agent — automatically pulls financial metrics, ratios, revenue trends, and key performance numbers on document ingestion.
- Build the Red Flag Agent — scans for anomalies, rising debt levels, falling margins, auditor qualifications, and unusual financial patterns automatically.
- Implement the multi-agent orchestration layer — coordinates Document, Extraction, and Red Flag agents sequentially on document upload.
- Test and validate agent outputs on seed financial documents for accuracy and source faithfulness.

27.3 Milestone 3 — Weeks 5–6: Research and Comparison

- Build the Research Agent — handles multi-part user queries with step-by-step retrieval, reasoning, and strict source citation.
- Build the Comparison Agent — cross-references extracted financial data across multiple uploaded company documents for side-by-side benchmarking.
- Develop the conversational research interface — users ask financial questions and receive cited, reasoned responses in a clean chat-style UI.
- Begin Report Agent development — define report structure and implement Executive Summary and Key Financials section generation.

27.4 Milestone 4 — Weeks 7–8: Reporting, Testing, and Launch

- Complete the Report Agent — compiles outputs from all agents into a structured PDF report covering Executive Summary, Key Financials, Red Flags, Company Comparison, and Outlook.
- Conduct end-to-end testing — validate citation accuracy, agent collaboration, extraction correctness, and report completeness.
- Optimize agent prompts, retrieval quality, and orchestration reliability across varied financial document formats.
- Prepare technical documentation, project report, and final demonstration.

27.5 Recommended Build Order

1. Create GitHub repository
2. Finalize project architecture
3. Set up FastAPI backend
4. Implement PDF upload
5. Extract text from PDFs
6. Chunk the extracted text
7. Generate embeddings
8. Store embeddings in ChromaDB
9. Build a basic RAG chatbot
10. Add citations using page metadata
11. Develop the Document Agent
12. Develop the Extraction Agent
13. Develop the Red Flag Agent
14. Develop the Comparison Agent
15. Develop the Research Agent
16. Develop the Report Agent
17. Integrate MongoDB for users, workspaces, chats, and reports
18. Build the React frontend
19. Connect frontend with backend APIs
20. Test, optimize, deploy, and document the project

28. Future Scope

Enterprise-grade capabilities considered valuable but explicitly deferred beyond the v1.0 internship deliverable.

- Real-time stock price and market data integration.
- Direct SEC / regulatory filing ingestion via official APIs.
- Financial forecasting and predictive modelling.
- Portfolio recommendation and allocation suggestions.
- Multi-language report generation and translation.
- Voice-based research assistant.
- Interactive, drill-down financial dashboards with live charting.
- Migration from LangChain's sequential pipeline to LangGraph for dynamic, non-linear agent collaboration.
- MongoDB Atlas Vector Search adoption to consolidate application data and embeddings into a single database.
- Role-based multi-tenant organization accounts for enterprise research teams.

29. Appendix

29.1 Glossary

Term	Definition
RAG	Retrieval-Augmented Generation — retrieving relevant source content before generating an answer, to ground the response in fact.
Embedding	A numerical vector representation of text that captures semantic meaning, enabling similarity search.
Chunk	A segment of a document's text, sized appropriately for retrieval and embedding.
Vector Database	A database optimized for storing and searching embeddings by similarity (e.g., ChromaDB).
Red Flag	An indicator of financial risk or concern, such as declining margins or rising debt.
Orchestration Layer	The component that decides which agent runs, in what order, and what data passes between agents.
MD&A	Management Discussion & Analysis — a narrative section of an annual report explaining financial results.
EPS / ROE / ROCE	Earnings Per Share / Return on Equity / Return on Capital Employed — standard financial ratios.

29.2 Reference Modules

- Research Workspace and Session Management Module
- Financial Document Ingestion and Parsing Module
- Embedding Generation and Vector Database Module
- Multi-Agent Orchestration Layer
- Financial Query and Conversational Research Interface
- Research Report Generation and PDF Export Module

29.3 Evaluation Criteria (Program-Defined)

- Accuracy of financial metrics extracted by the Extraction Agent and strict faithfulness to source documents.
- Quality and relevance of red flags and anomalies detected by the Red Flag Agent.
- Effectiveness of Research Agent query handling, multi-step reasoning, and source citation accuracy.
- Completeness and clarity of the final PDF report generated by the Report Agent.
- Smoothness of multi-agent collaboration, orchestration reliability, and overall system cohesion.
- Completeness of implementation, testing, documentation, and final demonstration.

29.4 Recommended Learning Path for the Team

For a team new to this domain, the following sequence mirrors how real-world AI teams build enterprise document intelligence systems:

1. Financial documents — understand what annual reports contain
2. LLMs and prompt engineering — know what the language model can and cannot do
3. Embeddings — learn how text is converted into vectors
4. Vector databases — understand semantic search
5. RAG — learn how retrieval and generation work together
6. Single-agent system — build one agent that answers questions from uploaded PDFs
7. Multi-agent orchestration — split responsibilities among specialized agents
8. Frontend and backend integration — connect the UI, APIs, and AI services
9. Testing and deployment — validate the system and make it usable

29.5 Document End

This PRD is a living document. As implementation progresses, sections such as Functional Requirements, API Design, and Milestones should be updated to reflect real engineering decisions, and the Revision History in Section 1.2 should be updated accordingly.